

torch.hamming_window#

https://pytorch.org/docs/stable/generated/torch.hamming_window.html#torch.hamming_window

Description:

Hamming window function. where NNN is the full window size. The input `window_length` is a positive integer controlling the returned window size. `periodic` flag determines whether the returned window trims off the last duplicate value from the symmetric window and is ready to be used as a periodic window with functions like `torch.stft()`. Therefore, if `periodic` is `true`, the NNN in above formula is in fact $\text{window_length} + 1$. Also, we always have `torch.hamming_window(L, periodic=True)` equal to `torch.hamming_window(L + 1, periodic=False)[: -1]`. If `window_length = 1`, the returned window contains a single value 1.

Function Signature

```
torch.hamming_window(window_length, *, dtype=None,
layout=None, device=None, pin_memory=False,
requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Returns

Return type

Parameters

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported. `layout` (`torch.layout`, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported. `device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`). `device` will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `pin_memory` (`bool`, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: `False`. `requires_grad` (`bool`, optional) – If autograd should record operations on

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported. `layout` (`torch.layout`, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported. `device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `pin_memory` (`bool`, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: `False`. `requires_grad` (`bool`, optional) – If autograd should record operations on

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported. `layout` (`torch.layout`, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported. `device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `pin_memory` (`bool`, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: `False`. `requires_grad` (`bool`, optional) – If autograd should record operations on

Keyword

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported. `layout` (torch.layout, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported. `device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False. `requires_grad` (bool, optional) – If autograd should record operations on

Returns:

Tensor

Notes & Warnings

NOTE

If `window_length = 1=1=1`, the returned window contains a single value 1.

NOTE

This is a generalized version of `torch.hann_window()`.

Detailed Documentation

Documentation

Hamming window function.

where NNN is the full window size.

The input `window_length` is a positive integer controlling the returned window size. `periodic` flag determines whether the returned window trims off the last duplicate value from the symmetric window and is ready to be used as a periodic window with functions like `torch.stft()`. Therefore, if `periodic` is `true`, the NNN in above formula is in fact $window_length+1 - window_length + 1 = window_length+1$. Also, we always have `torch.hamming_window(L, periodic=True)` equal to `torch.hamming_window(L + 1, periodic=False)[-1])`.

If `window_length = 1`, the returned window contains a single value 1.

This is a generalized version of `torch.hann_window()`.

`window_length` (int) – the size of returned window

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported.

`layout` (`torch.layout`, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported.

`device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`).

device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

A 1-D tensor of size $(\text{window_length},)$ containing the window.

Hamming window function with periodic specified.

`window_length` (int) – the size of returned window

`periodic` (bool) – If True, returns a window to be used as periodic function. If False, return a symmetric window.

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported.

`layout` (torch.layout, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported.

`device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned

memory. Works only for CPU tensors. Default: False.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

A 1-D tensor of size $(\text{window_length},)$ containing the window.

Hamming window function with periodic and alpha specified.

`window_length` (int) – the size of returned window

`periodic` (bool) – If True, returns a window to be used as periodic function. If False, return a symmetric window.

`alpha` (float) – The coefficient α in the equation above

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). Only floating point types are supported.

`layout` (torch.layout, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported.

`device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

`requires_grad` (bool, optional) – If autograd should record operations on the returned

tensor. Default: False.

A 1-D tensor of size $(\text{window_length},)(\text{window_length},)(\text{window_length},)$ containing the window.

Hamming window function with periodic, alpha and beta specified.

window_length (int) – the size of returned window

periodic (bool) – If True, returns a window to be used as periodic function. If False, return a symmetric window.

alpha (float) – The coefficient α in the equation above

beta (float) – The coefficient β in the equation above

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()). Only floating point types are supported.

layout (torch.layout, optional) – the desired layout of returned window tensor. Only torch.strided (dense layout) is supported.

device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

A 1-D tensor of size $(\text{window_length},)$ containing the window.